
Linux and Android Security Models Lab Material

Course: *Introduction to Vulnerability Research Lab*

Author name: *Dr. Bernard Lampe*

Due date: *December 5th, 2019*

Introduction

1. This lab pairs the **Linux and Android Security Models** lecture with two instruments: a runnable user-space model of a privileged service's enforcement gates (`model\_deputy.c`), and enumeration on a real host or device for the parts that need a live kernel.
2. Build the model (verified warning-free on the reference host):

```
1 clang -g -O0 -Wall -Wextra model_deputy.c -o model_deputy
```
3. The SELinux enumeration exercises need a Linux host with SELinux enabled (Fedora/RHEL/CentOS; on Debian/Ubuntu install `selinux-basics` and boot with `selinux=1`). Where no such host is available, do them as paper exercises against the AOSP `system/sepolicy` tree, which is the same data the tools read.

Exercise 1 — Audit the Model Deputy

1. Read `model\_deputy.c` as if its `deputy\_check/deputy\_use` pair were a privileged service's enforcement point (an LSM hook plus the operation it authorizes, or a Binder service method). Inventory: the subject, the object, the operation, and the policy data.
2. Identify the three planted model bugs *before* running anything. Each is one property of the reference monitor from the lecture:
 - (a) **incomplete mediation**: the MAC arm returns on the first rule whose domain matches and never consults the target's label, so every target inherits the first rule's verdict (a policy bug, not a memory-safety bug);
 - (b) **identity not attested**: the subject is built from the client-claimed uid/domain in the request rather than from the attested subject — the confused deputy's defining mistake;
 - (c) **check-then-use**: the path is resolved once for the check and again for the operation, so the object checked need not be the object used (the source auditing lecture's path race, in authorization form).
3. For each bug: which lecture family is it (enforcement-code, policy, or seam), and what must the attacker control to reach it?

Exercise 2 — Confirm Behaviorally

1. Run the built model. Verified reference behavior:

```

1 ok:                check=0 followed=0
2                use=app_data_file wrote="by deputy"
3 forged uid:    check=0
4                use=app_data_file wrote="by deputy"
5 wrong label:  check=0
6                use=system_data_file wrote="by deputy"
7 swap:         check=0 followed=0
8                use=system_data_file wrote="by deputy"

```

Exit status 0, and the transcript is byte-identical across runs. Read each record as: the check verdict, the label the use phase acted on, and the fact that the write happened (`wrote` carries the deputy's marker).

2. The correct verdicts, and why the model disagrees with all three:

Call	Model says	Should be	Denying layer
ok	0	0	—
forged uid	0	−2	DAC (uid 10001 is not owner 10002, no other bits)
wrong label	0	−3	MAC (untrusted_app may not write system_data_file)
swap	0	0	— (but the use must act on the object the check approved)

3. Explain the two easier lines in kernel terms. The `forged uid` call corresponds to a service that builds its subject from the request's uid field: the attested subject (app, uid 10001) is discarded, and the deputy is made to act as root. The `wrong label` call corresponds to a policy bug: the policy names `system\_data\_file` for `untrusted\_app`, but the check never reads the target, so the rule that should deny is unreachable.
4. The `swap` line is the seam bug, and it is the one that leaves no trace in the exit status: `check=0` reports success, `followed=0` reports that the check resolved a real file, and yet the use wrote `system\_data\_file`. A real kernel equivalent is a check on a path followed by a use that re-resolves the path after an attacker-controlled rename or symlink swap.
5. Note one model simplification, so the demo isolates the MAC arm: the system file is given mode 0666 rather than the real 0644. In a real system both arms deny that line; the model keeps only the MAC arm in play. State what changes in the table if the mode is 0644.
6. Fix each bug and re-run until the transcript matches the expected column:
 - (a) Bug 2: build the subject from attested, not from req (the comment marks the line).
 - (b) Bug 1: consult the target's label — match a rule only when both the domain and the target label match, and default to deny when no rule matches.
 - (c) Bug 3: resolve once, then act on the resolved object (pass the object, not the path, to the use phase), or re-validate the resolved object before acting.
7. For each fix, state which reference-monitor property it restores, and why a fuzzer would never have found any of the three.

Exercise 3 — Enumerate the Model on Linux

1. On the Linux reference host, establish the model layer by layer and fill the table. Use the lecture's **Enumerating the Model** recipes.

Operation	Denying layer	Recipe
read another user's 0600 file	DAC	<code>ls -l</code> , then read it as that user
write a file you do not own	DAC	<code>ls -l</code> ; compare owner and mode
read <code>/etc/shadow</code>	DAC	<code>ls -l /etc/shadow</code>
attach to a sibling process	Yama LSM	<code>cat /proc/sys/kernel/yama/ptrace_scope</code>
bind a port below 1024	capability	<code>capsh --print; getcap</code>
mount a filesystem	capability	<code>unshare -Ur -m id</code> , then mount
write a <code>/proc/sys</code> knob	MAC/DAC	<code>ls -Z</code> , then write it
read a file its label forbids	SELinux	<code>ls -Z; ps -eZ</code>

2. For two rows of the table, produce the actual denial. SELinux hosts log it; find it with `ausearch -m avc` (or the kernel log), then run `audit2allow -a` on the denial and read the `allow` rule it prints.
3. For each of those two denials, answer the model questions: which subject, which object, which operation, which rule *should* apply, and why the printed rule is over-broad. This is the policy-bug report in the lecture's **From Bug to Report** shape.
4. Pick one service domain that is permitted to act for a caller (a system service on the host, or `system\_server` on a device) and write its confused-deputy checklist answers from the lecture: the deputy, its authority, the attacker-controlled part of the request, and whether the check and the use name the same object.

Exercise 4 — Android Model Drills

1. With a device or emulator (a `userdebug` build is easiest), run the Android enumeration recipes from the lecture and answer, with the command output as evidence:
 - (a) `adb shell id` and `getenforce`: which uid is the shell, and which domain is it in?
 - (b) `ls -Z /data/data`: name two different app uids' directories and their labels. Why can the shell not read either, and which layer denies each (uid sandbox, SELinux, or both)?
 - (c) `ps -AZ`: find `system\_server`, `zygote`, and one vendor HAL domain. Which of them holds the most authority, and which of them is reachable from an app over Binder?
 - (d) `cmd package list permissions -g -d` and `dumpsys package <pkg>`: pick one dangerous and one signature permission and say who may hold each, and where the check is enforced.
 - (e) `service list`: pick one service and say whether an app can reach it, and which policy decides (`binder\_call`, `service\_manager`, or the permission check in the service).

2. Paper question, one paragraph: a service performs `checkCallingOrSelfPermission(P)` before writing a caller-supplied path. Write the subject, the deputy, the object, the intended rule, the enforced rule, and the primitive, exactly as the lecture's worked example does. Then answer: which layer should have stopped it, and why the MAC layer cannot.
3. Paper question, one paragraph: the bootloader is unlocked on a test device. Name the model layers that are now bypassable and the layers that are not, and say what an attacker gains and what they still cannot do.

Deliverables

1. Exercise 1's three bugs, each labelled by family (enforcement-code, policy, seam).
2. Exercise 2's expected-vs-observed table, the three fixes, and one line per fix naming the reference-monitor property restored.
3. Exercise 3's completed table, two real AVC denials with the `audit2allow` output, and the two policy-bug paragraphs.
4. Exercise 4's device evidence and the two model paragraphs (confused deputy and unlocked bootloader).